

30-Day Embedded C++ Learning Plan

Platform: Nordic nRF52840 | **RTOS:** Zephyr | **Toolchain:** GCC ARM / West / CMake

Reviewed by DeepSeek AI (rated 7/10 draft → revised to address all feedback).

Target learner: knows basic C, new to embedded C++ and Zephyr.

Plan Overview

Week	Theme
------	-------

|-----|-----|

Week 1	Foundations, Toolchain, Zephyr Setup, Debugging from Day 1
Week 2	Memory, Pointers, ARM Cortex-M Architecture, Hardware Interfacing
Week 3	OOP for Embedded, Driver Architecture, Zephyr Device Model
Week 4	RTOS Deep Dive, Modern Embedded C++, Power & Capstone

Week 1 — Foundations & Toolchain

Day	Topic	Key Concepts
----- ----- -----		
01	Intro to Embedded & Zephyr Toolchain	West, CMake, Kconfig, DTS, nRF52840 board target, first build
02	Bare Metal vs Zephyr Hello World	Zephyr `printf`, LED blink via GPIO API, `CMakeLists.txt`, `prj.conf`
03	Embedded Data Types	`uint8_t`, `uint32_t`, `stdint.h`, `volatile`, `const`, `static`, fixed-width literals
04	Bitwise Operations & Register Manipulation	AND/OR/XOR/shifts, bit masking, set/clear/toggle macros, nRF register access
05	Control Flow & Lookup Tables	`if/else`, `switch`, lookup tables as embedded optimization, `constexpr` tables
06	Functions & Function Pointers	Inline functions, function pointers, callbacks, Zephyr callback patterns
07	Debugging from Day 1	GDB + J-Link, RTT logging, Zephyr `LOG_*` macros, `__ASSERT`, `CONFIG_ASSERT`

Week 2 — Memory, Pointers & Hardware

Day	Topic	Key Concepts
-----	-------	--------------

|----|-----|-----|

08	ARM Cortex-M Architecture	Memory map, NVIC, SysTick, vector table, fault handlers, MPU, nRF52840 memory layout
09	Memory Segments & Linker Scripts	<code>`.text`</code> , <code>`.bss`</code> , <code>`.data`</code> , stack/heap, Zephyr linker script, map file analysis
10	Pointers & References for Hardware	MMIO via pointers, <code>`volatile`</code> pointers, <code>`const`</code> correctness, <code>`extern "C"``</code> linkage
11	Structs, Bit-fields & Unions	Register mapping structs, <code>`__packed`</code> , unions for type punning, Zephyr <code>`sys_`</code> structs
12	Arrays, Buffers & Static Allocation	Fixed-size arrays, <code>`std::array`</code> , <code>`K_MEM_POOL`</code> , avoiding heap in embedded
13	Interrupts & ISR Safety	Zephyr IRQ API, <code>`ISR_DIRECT_DECLARE`</code> , <code>`volatile`</code> in ISR context, <code>`irq_lock/unlock`</code>
14	GPIO & UART with Zephyr	<code>`gpio_pin_configure`</code> , <code>`uart_irq_callback_set`</code> , devicetree bindings, DTS overlays

Week 3 — OOP for Embedded & Driver Architecture

Day	Topic	Key Concepts
15	Classes & Objects in Embedded	Embedded OOP rules, no RTTI, no exceptions, <code>`nothrow`</code> , class size awareness
16		

|----|-----|-----|

	Constructors, Destructors & RAII	RAII for GPIO/DMA/mutexes, `ScopedLock`, deferred init pattern
17	Inheritance & HAL Design	Abstract base classes, virtual function vtable cost, Zephyr driver model analogy
18	Templates & Zero-Cost Abstractions	Template drivers, `static_assert`, CRTP pattern, policy-based design
19	Operator Overloading & Type-safe Registers	Register wrapper classes, type-safe GPIO pin classes, strong typedefs
20	Singleton & Static Peripheral Managers	Singleton for UART manager, Monostate pattern, `K_MUTEX_DEFINE` in C++
21	Layered Driver Architecture	HAL → Driver → Application, SPI/I2C Zephyr driver wrapper design

Week 4 — RTOS, Modern C++ & Capstone

Day	Topic	Key Concepts
-----	-------	--------------

|-----|-----|-----|

22	Error Handling in Embedded	Error codes, `static_assert`, `__ASSERT`, compile-time checks, no exceptions policy
23	Modern C++ in Embedded (C++11/14/17)	`constexpr`, `nullptr`, `noexcept`, scoped `enum class`, uniform initialization
24	Memory Management Advanced	Placement new, memory pools, `K_HEAP_DEFINE`, smart pointer trade-offs in embedded

25	State Machines	`enum class` FSM, template FSM, hierarchical state machines, Zephyr SMF framework
26	Ring Buffers & Lock-free Structures	Circular queue, `K_MSGQ_DEFINE`, lock-free ISR-to-task communication
27	Zephyr Threads & Scheduling	`K_THREAD_DEFINE`, priorities, cooperative vs preemptive, real-time constraints
28	Zephyr Synchronization Primitives	Mutexes, semaphores, condition vars with C++ RAIL wrappers, priority inversion
29	Power Management & Optimization	`pm_device`, sleep states, `CONFIG_PM`, compiler flags (`-Os`, LTO, gc-sections)
30	Capstone Project	UART command shell + LED PWM control + sensor polling — full OOP + Zephyr RTOS

Toolchain Requirements

```
# Install West (Zephyr meta-tool)
pip install west

# Initialize Zephyr workspace
west init ~/zephyrproject
cd ~/zephyrproject && west update

# Install Zephyr SDK (ARM toolchain)
wget https://github.com/zephyrproject-rtos/sdk-ng/releases/download/v0.16.8/zephyr-sdk-0.16.8-linux-x86_64.tar.xz
tar xf zephyr-sdk-0.16.8_linux-x86_64.tar.xz
cd zephyr-sdk-0.16.8 && ./setup.sh
```

```
# Install Python dependencies
pip install -r ~/zephyrproject/zephyr/scripts/requirements.txt
```

DeepSeek AI Review Summary

Original Rating: 7/10

Key improvements incorporated:

- Added ARM Cortex-M architecture day (Day 8)
- Moved debugging to Week 1 (Day 7) – debug from day one
- Moved dynamic memory to Week 4 (Day 24) – advanced topic
- Added error handling patterns day (Day 22)
- Added linker scripts day (Day 9)
- Fixed overloaded days (GPIO+UART split, RTOS split across days 27-28)
- Replaced generic RTOS with Zephyr-specific APIs throughout

- Added nRF52840 platform context to hardware days